

PromptBench

UCS 318 Software Engineering Project Report – Semester Evaluation

Submitted by:

Ridham Goyal (1024030417)

Niloy Sharma (1024030422)

Year: 2026–27 | Branch: COE
Group: 3C31

Submitted to:

Hardik Sir



Computer Science and Engineering Department
TIET, Patiala
2026

Acknowledgement

We take this opportunity to express our sincere gratitude to everyone who contributed to the successful completion of our project “PromptBench.”

First and foremost, we would like to extend our heartfelt thanks to Hardik Sir for continuous guidance, valuable suggestions, and constructive feedback throughout the duration of our project. Their encouragement and expertise played a pivotal role in helping us understand the technical challenges and refine our approach at every stage.

We are also grateful to the Thapar Institute of Engineering and Technology (TIET) for providing us with the necessary resources, infrastructure, and learning environment required to develop this project effectively. Their support enabled us to explore, design, and build a fully functioning system with confidence. Our sincere thanks also go to our peers and classmates for their insightful discussions, suggestions, and motivation during the development of this work. Lastly, we would like to express our deep appreciation to our families for their constant encouragement, patience, and belief in us.

We wholeheartedly dedicate this project to everyone who supported us, guided us, and stood by us in completing PromptBench successfully.

S.No.	Student Name	Roll Number
1	Ridham Goyal	1024030417
2	Niloy Sharma	1024030422

Contents

1	Project Selection Phase	3
1.1	Software Bid	3
1.2	Project Overview	4
2	Analysis Phase	5
2.1	Use Case	5
2.1.1	Use-Case Diagram	5
2.1.2	Use-Case Templates	6
2.2	Activity Diagram and Swimlane Diagram	10
2.3	Data Flow Diagrams (DFDs)	12
2.3.1	DFD Level 0	12
2.3.2	DFD Level 1	12
2.3.3	DFD Level 2	12
2.4	Software Requirement Specification (IEEE Format)	13
2.4.1	Introduction	13
2.4.2	Overall Description	14
2.4.3	Specific Requirements	14
2.5	User Stories and User Cards	16
2.6	Entity-Relationship Diagram	17
2.7	Class Diagram	17

1. Project Selection Phase

In this phase, the final project idea is chosen based on feasibility, requirements, and expected outcomes. This provides a clear foundation for preparing the software bid.

1.1 Software Bid

Please enter the names of your preferred team members:

- You are required to form a two to four person team.
- Choose your team members wisely. You will not be allowed to change teams.

Name	Roll No	Project Experience	Programming Language used	Signature
Ridham Goyal	1024030417	[Describe relevant project experience]	[Languages used]	
Niloy Sharma	1024030422	[Describe relevant project experience]	[Languages used]	

Programming Language / Environment Experience

List the languages you are most comfortable developing in, as a team, in your order of preference:

1. Python
2. JavaScript

and further languages

Choices of Projects

	Project Name	Unique Selling Point
First Choice	PromptBench	Most coding-assessment platforms grade whether a candidate can write an algorithm. PromptBench instead grades a skill that matters just as much on the job today: working effectively with an AI assistant to solve an open-ended business problem. Candidates investigate a case by chatting with a Helper AI, then submit a final recommendation that is privately graded by a separate Judge AI against a hidden rubric and ground-truth notes – without ever exposing the grading criteria to the Helper AI itself.
Second Choice	[Project name]	[Describe USP]
Third Choice	[Project name]	[Describe USP]
Fourth Choice	[Project name]	[Describe USP]

Additional Remarks / Inputs

The development of PromptBench provided valuable insight into designing systems where an AI component must be trusted for one purpose (assisting) but explicitly distrusted for another (grading). The backend – authentication, case-study delivery, the Helper AI chat loop, the Judge AI grading pipeline, and anti-gaming checks – is complete and covered by an end-to-end automated test suite. The React frontend is now also complete, with all planned pages (login,

signup, case list, case workspace, score dashboard, attempt history) wired to the live backend. The main items remaining are deployment of the frontend and backend, and expanding the seed content from 5 to the 8–10 case studies targeted for “feature complete.”

1.2 Project Overview

Purpose

The main goal of PromptBench is to:

- Provide realistic, open-ended business case studies for practicing AI-assisted problem solving.
- Let a candidate work through a case with a conversational Helper AI before committing to an answer.
- Grade the candidate’s process, not just their final answer, using a private Judge AI and a hidden rubric.
- Detect attempts to game the evaluation, such as submitting a pre-written answer with no real investigation.

Target Users

- **Candidates** – register, attempt case studies, and review their scored history.
- **Helper AI** – a scoped, candidate-facing assistant with access only to public case context.
- **Judge AI** – a server-side evaluator invoked only at submission time.

Key Features

- Secure signup/login with JWT-protected sessions and bcrypt-hashed passwords.
- A library of business case studies spanning three categories: data diagnosis, strategy generation, and customer reasoning.
- A persisted chat workspace where every candidate message and Helper AI reply is stored as a transcript turn.
- A one-shot Judge AI grading call that returns a structured, rubric-based score with reasoning.
- Automated anti-gaming checks (near-zero conversation, too-fast submission, pre-written-answer detection).
- A personal history dashboard of past attempts and scores.

System Overview

PromptBench operates as a web-based platform where:

1. A candidate creates an account and logs in.
2. The candidate browses the available business case studies.
3. The candidate opens a Case Workspace, reviews the case context, and interacts with the Helper AI.
4. The candidate drafts and submits a final recommendation.
5. The Judge AI privately grades the full transcript and final answer against the hidden rubric and ground truth.
6. The candidate reviews their overall score, per-dimension breakdown, and reasoning, and can revisit past attempts at any time.

2. Analysis Phase

2.1 Use Case

Actors

- Candidate
- Helper AI (system actor)
- Judge AI (system actor)

Candidate Use Cases

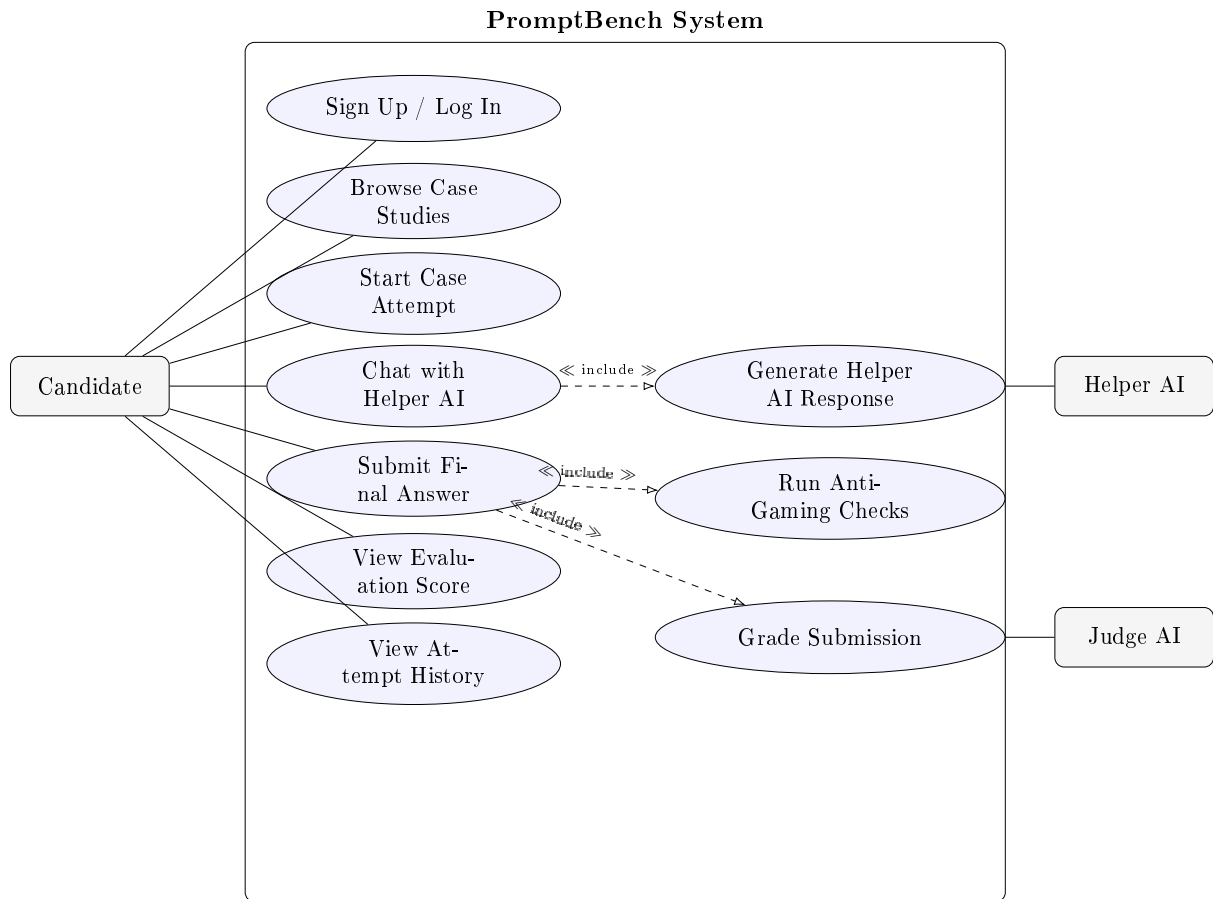
1. Sign Up / Log In
2. Browse Case Studies
3. Start a Case Attempt
4. Chat with Helper AI
5. Draft and Submit Final Answer
6. View Evaluation Score
7. View Attempt History

System Actor Use Cases

8. Generate Helper AI Response
9. Run Anti-Gaming Checks
10. Grade Submission

2.1.1 Use-Case Diagram

The use-case diagram of the PromptBench system illustrates the interactions between the Candidate and the two AI actors – Helper AI and Judge AI – and the system’s functionalities. A candidate can sign up, log in, browse case studies, start an attempt, chat with the Helper AI, and submit a final answer. Submitting a final answer includes running anti-gaming checks and invoking the Judge AI, which never interacts with the candidate directly. The Helper AI is only ever invoked while a chat message is being sent, and never has access to the grading path.



2.1.2 Use-Case Templates

Use Case: Sign Up / Log In

Actor: Candidate

Description: Candidate creates an account or authenticates to obtain a session token.

Preconditions: None

Postconditions: Candidate holds a valid JWT and can access protected routes.

Main Flow:

1. Candidate submits email, password (and name, if signing up).
 2. System validates credentials / uniqueness of email.
 3. System hashes and stores the password (signup) or verifies it (login).
 4. System issues a signed JWT on successful login.
-

Use Case: Browse Case Studies

Actor: Candidate

Description: Candidate views the list of available business case studies.

Preconditions: Candidate is authenticated.

Postconditions: List of case studies (title, category, difficulty) is displayed.

Main Flow:

1. Candidate opens the question list page.
2. System fetches all questions from the database.
3. System returns title, category, and difficulty for each case.

Use Case: Start Case Attempt

Actor: Candidate

Description: Candidate begins a new attempt on a chosen case study.

Preconditions: Case study exists.

Postconditions: A new submission record is created in the `in_progress` state.

Main Flow:

1. Candidate selects a case study.
2. System verifies the case exists.
3. System creates a submission owned by the candidate.

Relationship: Precedes → Chat with Helper AI

Use Case: Chat with Helper AI

Actor: Candidate

Description: Candidate exchanges messages with the Helper AI while investigating the case.

Preconditions: Submission is in progress and owned by the candidate.

Postconditions: Candidate and assistant turns are persisted to the transcript.

Main Flow:

1. Candidate sends a message.
2. System stores the candidate's turn.
3. System calls the Helper AI with the public case context and full chat history.
4. System stores the Helper AI's reply and returns it to the candidate.

Relationship: Includes → Generate Helper AI Response

Use Case: Submit Final Answer

Actor: Candidate

Description: Candidate submits their final recommendation for grading.

Preconditions: Submission is in progress (or a previously failed evaluation is being retried).

Postconditions: Submission is marked `submitted` and an evaluation record is created.

Main Flow:

1. Candidate submits final answer text.
2. System runs anti-gaming checks against the transcript and timing.
3. System invokes the Judge AI with the transcript, final answer, rubric, and ground truth.
4. System validates the Judge AI's JSON response and stores the evaluation.

Relationship: Includes → Run Anti-Gaming Checks, Grade Submission

Use Case: View Evaluation Score

Actor: Candidate

Description: Candidate reviews the score breakdown for a graded submission.

Preconditions: Submission status is **submitted**.

Postconditions: Overall score, per-dimension scores, reasoning, and any anti-gaming flags are displayed.

Main Flow:

1. Candidate opens the results page for a submission.
 2. System verifies ownership and grading status.
 3. System returns the evaluation record and transcript.
-

Use Case: View Attempt History

Actor: Candidate

Description: Candidate views a list of all previously graded attempts.

Preconditions: Candidate is authenticated.

Postconditions: A chronological list of past submissions and scores is displayed.

Main Flow:

1. Candidate opens the attempts page.
 2. System fetches all **submitted** submissions owned by the candidate, joined with their evaluations.
-

Use Case: Generate Helper AI Response

Actor: Helper AI (system)

Description: System generates a Helper AI reply scoped only to public case context.

Preconditions: A candidate message has been received.

Postconditions: A reply is generated without any access to the rubric or ground truth.

Main Flow:

1. System builds a system prompt from the case's public context only.
2. System sends the conversation history to the Claude API.
3. System returns the assistant's reply.

Use Case: Run Anti-Gaming Checks

Actor: System

Description: System evaluates the submission for signs of gaming the assessment.

Preconditions: Final answer has been submitted.

Postconditions: A list of warning flags (possibly empty) is attached to the evaluation.

Main Flow:

1. Check transcript length (near-zero conversation).
2. Check elapsed time between start and submission (too-fast submission).
3. Check similarity between the first candidate message and the final answer (pre-written answer).

Use Case: Grade Submission

Actor: Judge AI (system)

Description: System privately grades the candidate's process and final answer.

Preconditions: Final answer has been submitted; anti-gaming checks have run.

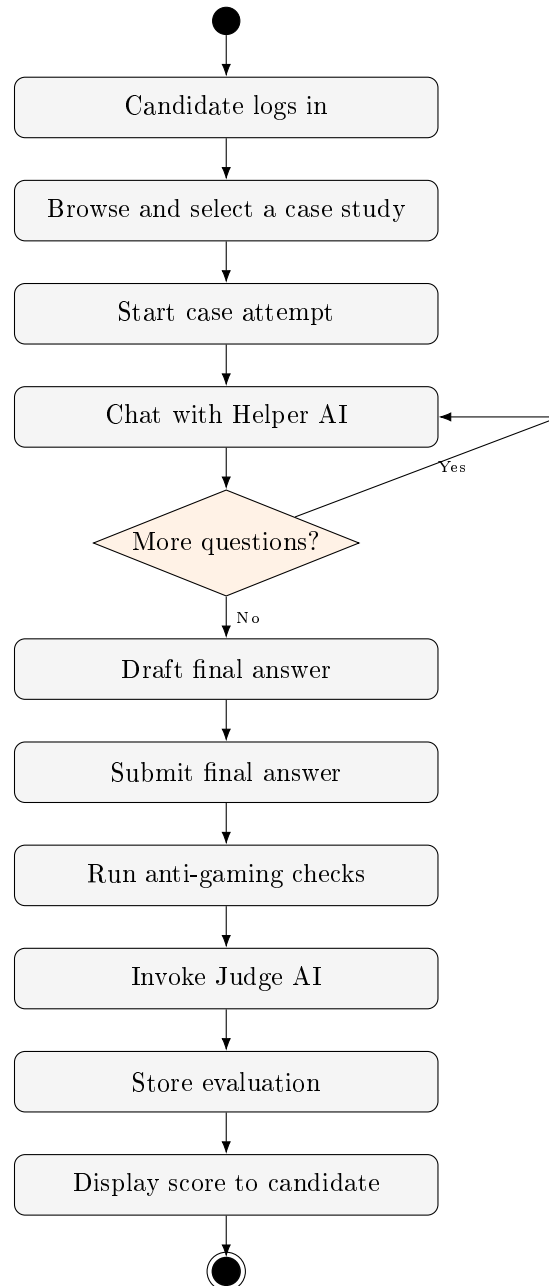
Postconditions: A structured JSON report (overall score, dimension scores, reasoning) is validated and stored.

Main Flow:

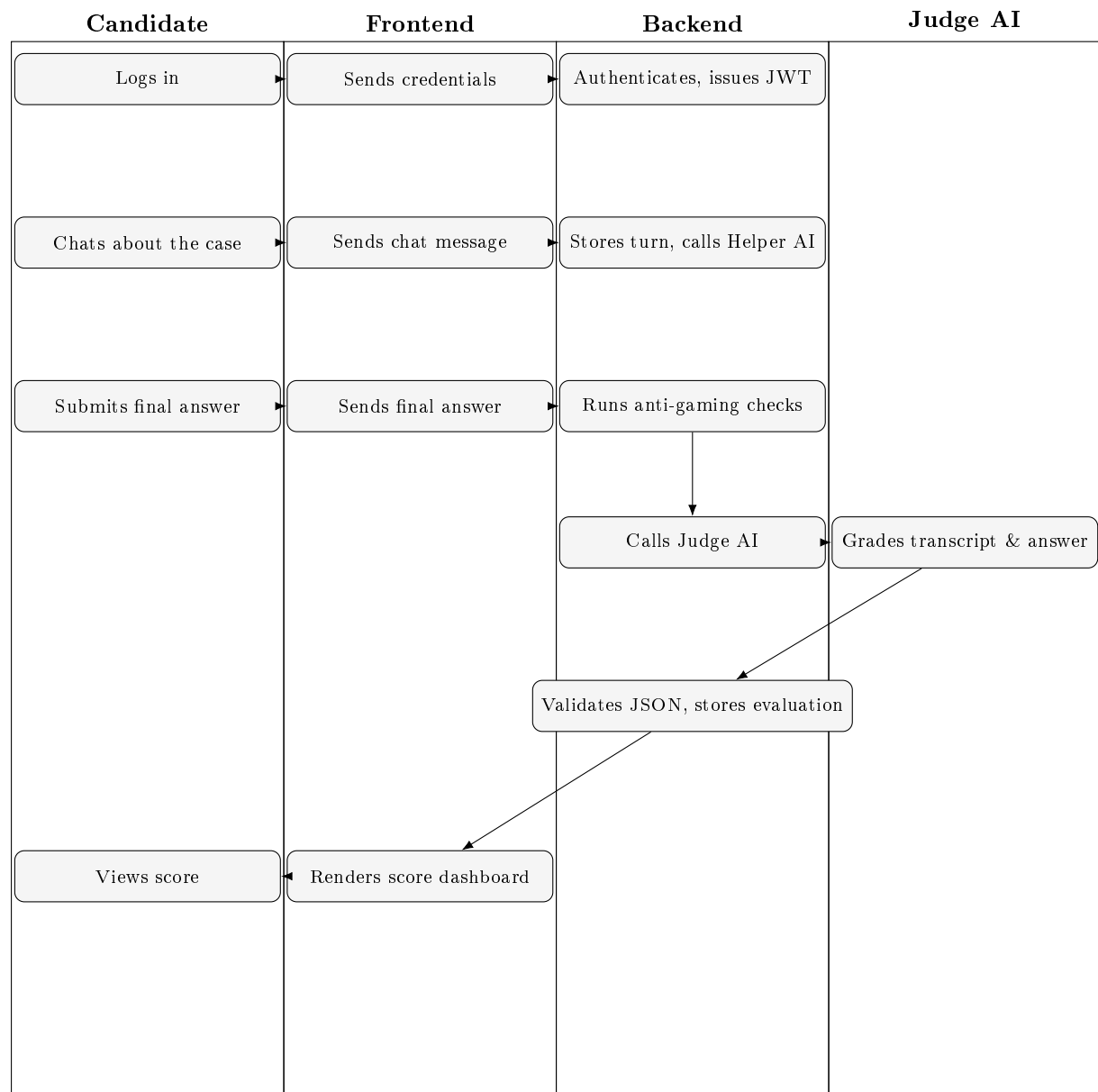
1. System builds a system prompt containing the rubric and hidden ground-truth notes.
2. System sends the full transcript and final answer to the Claude API.
3. System parses and validates the returned JSON against a strict schema.
4. System stores the evaluation and marks the submission as **submitted**.

2.2 Activity Diagram and Swimlane Diagram

The activity diagram of the PromptBench system illustrates the complete workflow of a candidate's attempt, from login through to receiving a graded score. The candidate logs in, selects a case, and works through it in conversation with the Helper AI. On submission, the system runs anti-gaming checks and invokes the Judge AI, which returns a structured score that the candidate can then review.



This swimlane diagram illustrates the same workflow split across the Candidate, the Frontend, the Backend, and the Judge AI, showing where each responsibility sits: the frontend only relays candidate actions, the backend owns persistence and orchestration, and the Judge AI is reached only once, at the very end.

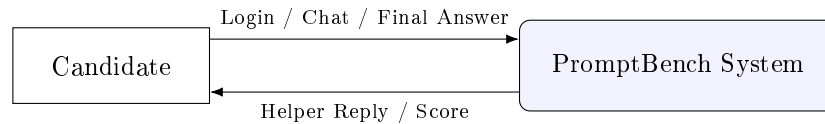


2.3 Data Flow Diagrams (DFDs)

The following figures illustrate the flow of information within PromptBench, from a high-level context down to detailed process-level data movement.

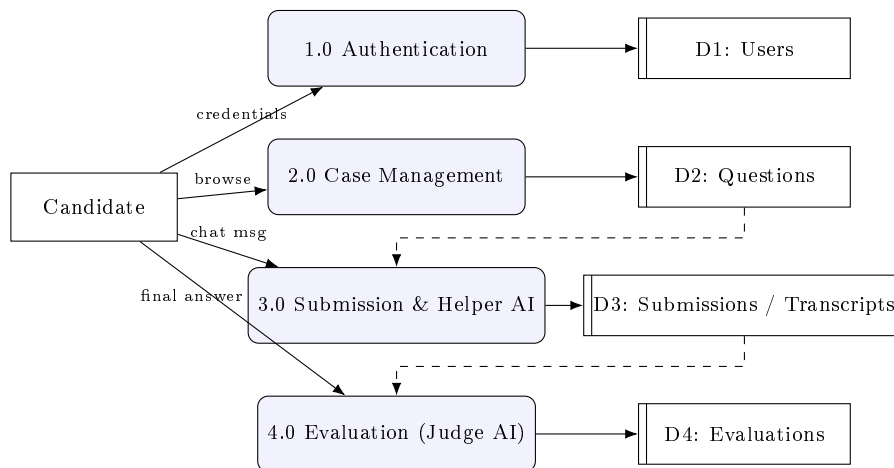
2.3.1 DFD Level 0

This Level 0 diagram provides a high-level overview of the PromptBench system and its single external entity, the Candidate. The candidate sends chat messages and a final answer, and receives Helper AI replies and a graded score in return.



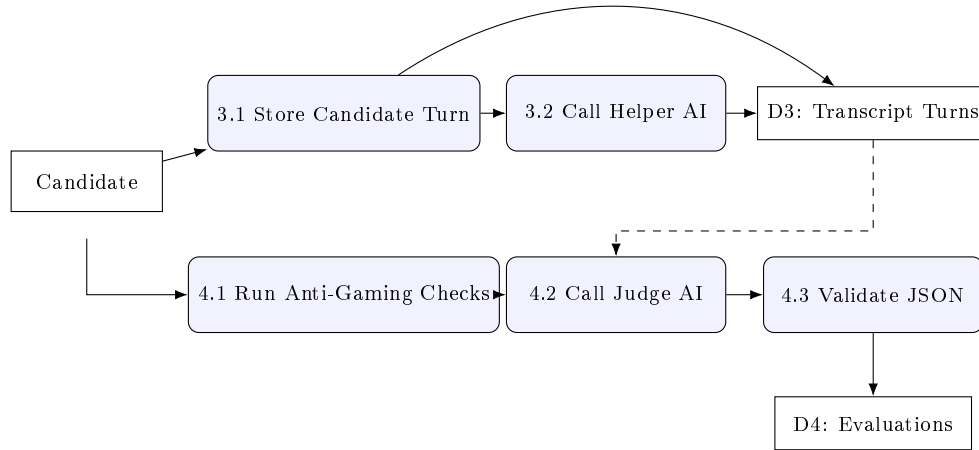
2.3.2 DFD Level 1

This Level 1 diagram breaks the system into its four major processes: Authentication, Case Management, Submission & Helper AI Interaction, and Evaluation. Each process reads from or writes to its corresponding data store.



2.3.3 DFD Level 2

This Level 2 diagram expands processes 3.0 and 4.0, showing how a chat message becomes a stored transcript turn and Helper AI reply, and how a final answer flows through anti-gaming checks and the Judge AI into a stored evaluation.



2.4 Software Requirement Specification (IEEE Format)

PromptBench

2.4.1 Introduction

Purpose

The purpose of this document is to provide a detailed description of the PromptBench system, a platform designed to let candidates practice and be evaluated on their ability to solve open-ended business problems in collaboration with an AI assistant. This document serves as a reference for developers (system design and implementation), instructors (evaluation criteria), and candidates (end users).

Scope

PromptBench is a web-based system that:

- Presents business case studies across three categories: data diagnosis, strategy generation, and customer reasoning.
- Provides a Helper AI for candidates to converse with while investigating a case.
- Grades the full transcript and final answer using a private Judge AI against a hidden rubric and ground truth.
- Flags likely attempts to game the assessment.

Definitions, Acronyms, Abbreviations

- **Helper AI:** candidate-facing assistant with access only to public case context.
- **Judge AI:** server-side evaluator with access to the rubric and ground truth.
- **Rubric:** weighted scoring dimensions used by the Judge AI.
- **Ground truth notes:** hidden reference notes never shown to the candidate or Helper AI.
- **JWT:** JSON Web Token, used for authenticating API requests.
- **DB:** Database.

References

- IEEE SRS Standards.
- Software Engineering course material.

Overview

This document describes system features, functional and non-functional requirements, user roles, and system constraints.

2.4.2 Overall Description

Product Perspective

PromptBench is a standalone web application consisting of a React frontend, a FastAPI backend, a PostgreSQL database, and an external AI provider (Claude API) used for both the Helper AI and Judge AI roles.

Product Functions

Candidate functions: sign up / log in, browse case studies, start an attempt, chat with the Helper AI, submit a final answer, view evaluation score, view attempt history.

System functions: generate Helper AI responses, run anti-gaming checks, grade submissions via the Judge AI.

User Characteristics

- **Candidate:** basic technical/business literacy, uses the system to practice a skill.
- **Helper AI / Judge AI:** automated system actors, no human interaction.

Constraints

- Requires an internet connection and a configured Claude API key.
- Currently targets local development; no deployment configuration exists yet.

Assumptions and Dependencies

- Users have basic digital literacy.
- The Claude API is available and returns well-formed responses (or the system falls back to mocked responses in test mode).

2.4.3 Specific Requirements

External Interface Requirements

User Interface: simple login/signup forms, a case list, a chat-style case workspace, and a score dashboard.

Software Interface: PostgreSQL database, Anthropic Claude API.

Functional Requirements

FR1 – Sign Up / Log In: Input: email, password (+ name). Output: account created / JWT issued.

FR2 – List Case Studies: Output: list of questions with title, category, difficulty.

FR3 – Start Submission: Input: question id. Output: new submission id.

FR4 – Send Chat Message: Input: message text. Output: Helper AI reply, persisted transcript turn.

FR5 – Submit Final Answer: Input: final answer text. Output: structured evaluation (overall score, dimension scores, reasoning).

FR6 – Run Anti-Gaming Checks: Output: list of warning flags attached to the evaluation.

FR7 – View Evaluation: Output: full evaluation report and transcript for a submission.

FR8 – View Attempt History: Output: list of past graded submissions and scores.

Performance Requirements

- API responses (excluding LLM calls) should return in under 1 second under normal load.
- The system should support multiple concurrent candidates without cross-contaminating sessions.

Logical Database Requirements

The system includes five entities: User, Question, Submission, TranscriptTurn, and Evaluation

(see the Entity-Relationship Diagram in Section 2.6). Each entity has a unique identifier and well-defined relationships with the others.

Quality Attributes

- **Usability:** simple, chat-first workspace.
- **Reliability:** retry-safe JSON parsing for the Judge AI's structured output.
- **Scalability:** stateless JWT auth allows horizontal scaling of the backend.
- **Security:** bcrypt password hashing, JWT-protected routes, strict ownership checks on every submission and evaluation endpoint.
- **Integrity:** architectural isolation between the Helper AI and Judge AI.

Other Requirements

Future Enhancements: configurable rubric schemas per category, an admin interface for authoring case studies, deployment to a cloud platform, and score-weighted anti-gaming penalties.

2.5 User Stories and User Cards

User Stories – PromptBench

1. As a candidate, I want to browse business case studies so that I can choose one to practice.
2. As a candidate, I want to chat with a Helper AI so that I can investigate the case before answering.
3. As a candidate, I want to submit a final recommendation so that my work can be graded.
4. As a candidate, I want to see a per-dimension score breakdown so that I understand exactly where I did well or fell short.
5. As a candidate, I want to review my past attempts so that I can track my improvement over time.
6. As the Judge AI, I need the rubric and hidden ground truth so that I can grade objectively without ever exposing them to the candidate.
7. As the system, I want to flag near-zero-effort or pre-written submissions so that the evaluation stays meaningful.

Story Cards

Story Card 1

Title: Browse Case Studies

Actor: Candidate

Description: Candidate views the list of available case studies.

Acceptance Criteria:

- System displays title, category, and difficulty for each case.
- Hidden fields (rubric, ground truth) are never included in the response.

Story Card 2

Title: Chat with Helper AI

Actor: Candidate

Description: Candidate exchanges messages with the Helper AI.

Acceptance Criteria:

- Every message and reply is persisted in order.
- The Helper AI only receives the public case context, never the rubric.

Story Card 3

Title: Submit Final Answer

Actor: Candidate

Description: Candidate submits their final recommendation for grading.

Acceptance Criteria:

- Anti-gaming checks run automatically at submission time.
- The Judge AI's structured response is validated before being stored.

Story Card 4

Title: View Evaluation Score

Actor: Candidate

Description: Candidate reviews the graded result of a submission.

Acceptance Criteria:

- Overall score, dimension scores, reasoning, and any anti-gaming flags are shown.

- Only the owning candidate can view the evaluation.

Story Card 5

Title: View Attempt History

Actor: Candidate

Description: Candidate views a history of past graded attempts.

Acceptance Criteria:

- List is ordered by most recent submission first.
- Only `submitted` (fully graded) attempts are shown.

Story Card 6

Title: Grade Submission

Actor: Judge AI

Description: System grades the transcript and final answer against a hidden rubric.

Acceptance Criteria:

- Returned JSON is validated against a strict schema before being stored.
- On failure, the submission is marked `evaluation_failed` and can be retried.

Story Card 7

Title: Run Anti-Gaming Checks

Actor: System

Description: System flags submissions that show signs of gaming the assessment.

Acceptance Criteria:

- Near-zero conversation, too-fast submission, and pre-written-answer patterns are each independently detected.
- Flags are stored alongside the evaluation, not used to block submission.

2.6 Entity-Relationship Diagram

The Entity-Relationship diagram of the PromptBench system captures its five core entities – `USER`, `QUESTION`, `SUBMISSION`, `TRANSCRIPT_TURN`, and `EVALUATION` – along with an embedded `DIMENSION_SCORES` structure. A `USER` creates many `SUBMISSION`s, and each `SUBMISSION` is tied to one `QUESTION`. A `SUBMISSION` in turn contains many `TRANSCRIPT_TURN` records (the full chat history) and is evaluated by exactly one `EVALUATION`, which embeds the four rubric dimension scores produced by the Judge AI. This structure directly mirrors the SQLAlchemy models used in the backend.

2.7 Class Diagram

The class diagram of the PromptBench system represents its static structure: the persistent domain classes (`User`, `Question`, `Submission`, `TranscriptTurn`, `Evaluation`) alongside the three service-level classes that implement the system’s core behaviour (`HelperAIService`, `JudgeAIService`, `AntiGamingChecker`). The service classes operate on domain objects but are not themselves persisted.

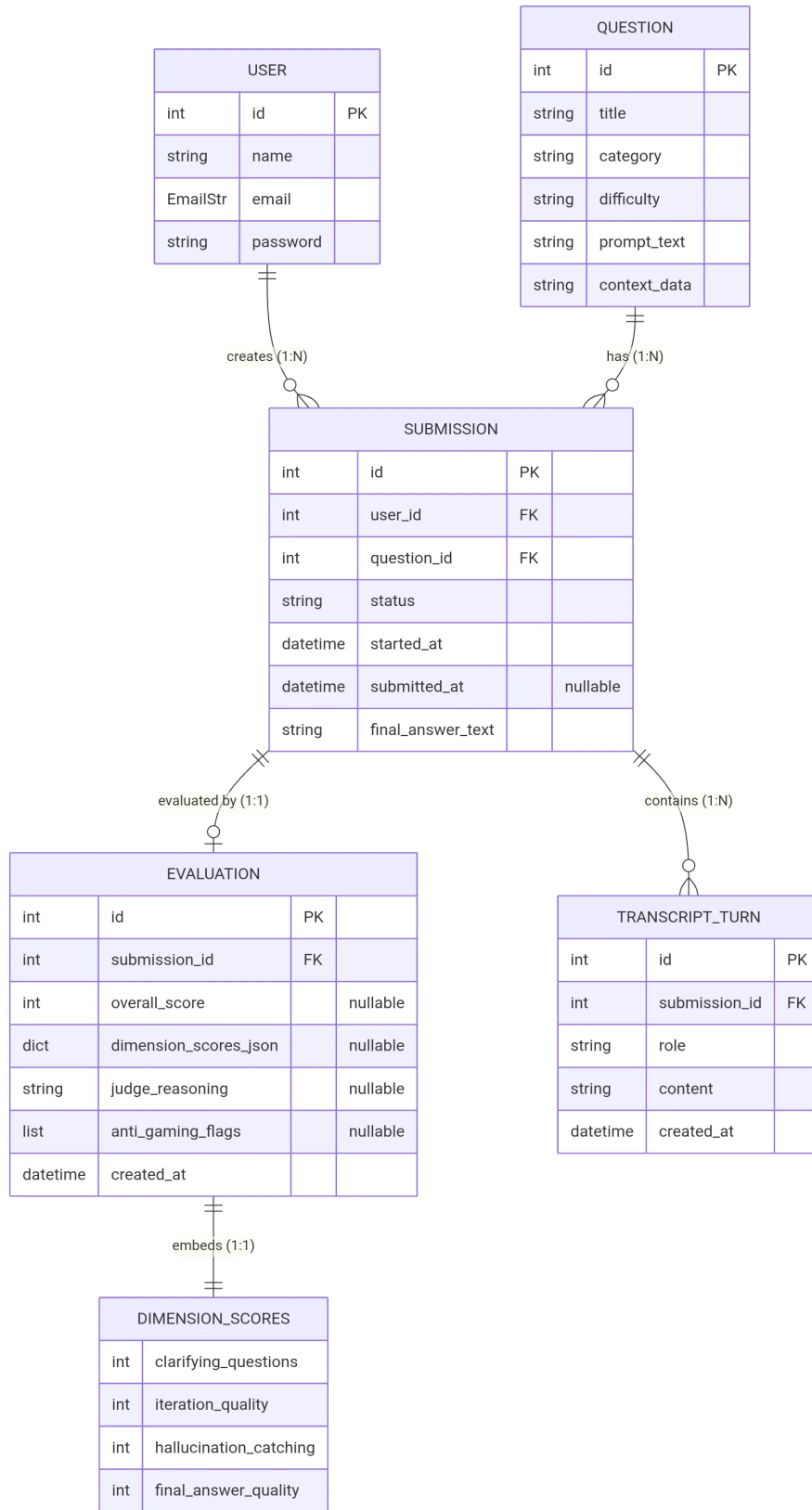


Figure 1: Entity-Relationship Diagram – PromptBench

